

Memòria de la Pràctica 1

Agents intel·ligents adversarials i basats en cerques

Serafí Nebot Ginard
Jaume Galmés Ramis

21722 - Intel·ligència Artificial
Grau d'Enginyeria Informàtica
Curs 2025/2026

Índex

1	Introducció	1
2	Estructura de dades	2
2.1	Classe Pos i tractament de la percepció	2
2.2	Classe Estat	2
3	Algorismes basats en cerca	4
3.1	Algorisme Depth-First Search	4
3.2	Algorisme A*	4
4	Algorisme adversarial - Minimax	5
4.1	Plantejament	5
4.2	Poda α - β	5
4.3	Execució	6
5	Estudi dels resultats	7
5.1	Mesura de rendiment: Temps d'execució	7
5.2	Mesura de rendiment: Nombre de passes	8
6	Conclusió	10

Introducció

Aquesta memòria pretèn mostrar la feina realitzada sobre la pràctica 1, en les següents pàgines s'exposa l'abordament del problema des del disseny de l'estructura de dades, passant per el plantejament i implementació dels distints algorismes i acabant amb l'estudi dels resultats obtinguts.

Estructura de dades

Dins aquest apartat es descriuen les estructures de dades que hem considerat adients per a fonamentar un problema com el proposat, entenent la seva naturalesa i les particularitats del mateix.

2.1 Classe Pos i tractament de la percepció

A partir de les dades proporcionades per el mètode `percepció` de `joc.py` podem estructurar les dades efectives, és a dir, les que ens seràn útils per a poder dur a terme la dinàmica del joc.

Per conveniència s'ha creat una classe `Pos` (dins `estat.py`) que representa una tupla amb les coordenades `x`, `y` d'una posició del tauler. S'han implementat uns quants *magic methods* per a poder realitzar operacions aritmètiques i lògiques bàsiques de forma fàcil. `Pos` també hereta de `NamedTuple` per a simplificar l'assignació dels atributs i fer-los immutables.

Per tant, dins del nostre codi, algunes de les percepcions com la dimensió o el torn es mantenen iguals, mentre que altres percepcions com la posició dels agents i el destí es converteixen en objectes `Pos`, de forma idèntica succeeix amb les parets, on aquestes formen un conjunt d'objectes `Pos`.

2.2 Classe Estat

Dins l'arxiu `estat.py` trobam les classes `Estat` i `EstatAdv`, aquestes formulen els estats i permeten donar una representació lògica de l'entorn en un moment concret. La implementació de dues classes separades ve justificada principalment per qüestions de facilitat. D'aquesta forma entenem `Estat` com una representació d'un sol agent sobre el tauler, mentre que l'ús d'atributs addicionals donen a `EstatAdv` la capacitat de treballar sobre un entorn multiagent.

Un estat sobre el problema donat, en la seva forma més atòmica, es compon dels següents atributs:

- La posició del/s agent/s sobre el tauler
- La posició de destí final (objectiu)
- La posició dels obstacles (parets)

Addicionalment trobem atributs derivats de la naturalesa de l'algorisme amb el qual estem treballant, necessaris per a poder resoldre el problema, alguns d'ells són:

- El camí com a seqüència d'estats i accions que duen fins al node en qüestió
- La heurística de cada agent
- El cost del camí acumulat
- El torn de l'agent que actua sobre l'estat

Així mateix dins les classes d'estats hi trobam la definició de les accions que l'agent pot dur a terme, juntament amb el seu respectiu cost. Aquestes entren en joc als mètodes involucrats amb la generació de fills i el seu camí associat. Mètodes com `accio` s'encarrèguen d'aplicar una acció sobre un estat donat, retornant l'estat consegüent, mentre que d'altra banda els mètodes `fills` empren aquesta per a generar una llista de tots els possibles estats resultants.

També trobam mètodes com `meta`, encarregat de comprovar si en l'estat actual un agent ha arribat al destí o si no hi ha moviments disponibles, i funcions auxiliars com `hash` i `eq`, que permeten treballar amb els estats dins d'estructures de dades.

Algorismes basats en cerca

Els següents algorismes basats en cerca permeten, donat un estat inicial, obtenir com a solució un camí

3.1 Algorisme Depth-First Search

L'algorisme de cerca en profunditat o *Depth-First Search* (DFS) és un tipus de cerca no informada que recorre de forma iterada el camí descendent de pares a fills. Dins el nostre projecte aquest s'ha implementat emprant principalment un llistat dels nodes oberts i un conjunt dels nodes tancats, mitjançant l'extracció i posterior visita del darrer element (-1) de la llista de nodes oberts. A cada iteració es comprova si el node fill és l'estat objectiu.

3.2 Algorisme A*

Per contraposició a l'algorisme DFS, l'A* és un algorisme de cerca informada, és a dir, a cada iteració, s'avalua la bondat d'un node tenguent en compte no només la seva heurística sino també el cost acumulat fins al mateix.

Per a dur a terme la bondat d'un estat s'ha emprat com a heurística la distància Manhattan, és a dir:

Sigui $\mathbf{p}$ la posició de l'agent a l'estat $\mathbf{n}$ i $\mathbf{d}$ la posició de l'objectiu, amb coordenades:

$$\mathbf{p} = \begin{bmatrix} x_p & y_p \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} x_d & y_d \end{bmatrix}$$

Obtenim l'heurística com:

$$h(\mathbf{n}) = d(\mathbf{p}, \mathbf{d}) = \sum_{i=1}^2 |p_i - d_i| = |x_p - x_d| + |y_p - y_d|$$

Algorisme adversarial - Minimax

A diferència dels algorismes de cerca informats/no informats, els algorismes adversarials ens permeten avaluar cada jugada on la solució no és el camí sino la acció òptima.

4.1 Plantejament

L'algorisme `minimax` genera, a partir d'un estat donat, totes les possibles jugades, avaluant la bondat de cada una i finalment retornant la que resulta més adient per a l'agent del torn actual. Recordem que ens interessa que l'agent MAX dugui a terme el moviment que li pugui donar més avantatge fent el supòsit de que MIN actuarà de la forma més obstructiva possible, d'aquesta forma asseguram que `minimax` es posi en el pitjor dels casos i no només contempli quan l'agent MIN jugui de forma subòptima.

Per a implementar aquest comportament al codi, s'ha definit que la puntuació que avalua una jugada sigui com de molt s'ha aconseguit obstruir a l'agent contrari, és a dir:

Siguin a_{act} i a_{adv} l'agent del torn actual i el seu adversari, i $d(a)$ la distància Manhattan d'un agent a respecte al destí:

$$\text{puntuació}(a_{act}) = \begin{cases} d(a_{adv}), & \text{si } d(a_{act}) = 0, \\ -d(a_{act}), & \text{si } d(a_{adv}) = 0, \\ 0, & \text{en cas contrari.} \end{cases}$$

4.2 Poda α - β

La poda α - β permet reduir el nombre de nodes explorats. Dins el nostre projecte aquesta es veu reflexada com a dos paràmetres de la funció `minimax`.

La primera crida a `minimax` es produeix al mètode `actua` on aquest inicialitza $\alpha = -\infty$ i $\beta = +\infty$. Posteriorment les crides recursives al mètode `minimax` van confinant l'interval entre α - β fins a arribar, si s'esdevé, al compliment de la condició de poda ($\alpha \geq \beta$).

4.3 Execució

A continuació es presenta una descripció general del funcionament de l'algorisme **minimax**, especificant les passes que segueix durant l'execució.

A partir d'un estat passat per paràmetre:

Algorisme Minimax

Pas 1: Inicialitza $\alpha = -\infty$ i $\beta = +\infty$.

Pas 2: Si l'estat actual és un estat final, es retorna a ell mateix (conté la puntuació).

Pas 3: Genera tots els estats fills possibles.

Pas 4: Per a cada fill:

Pas 4:1. Calcula recursivament el valor de **minimax**.

Pas 4:2. Actualitza α o β segons el torn.

Pas 4:3. Si $\alpha \geq \beta$, aplica la poda i surt del bucle.

Pas 5: Retorna el valor màxim o mínim segons l'agent del torn.

Estudi dels resultats

A partir dels algorismes implementats, exposam dins aquest apartat els resultats obtinguts després d'haver duit a terme l'execució repetida dels 3 algorismes.

5.1 Mesura de rendiment: Temps d'execució

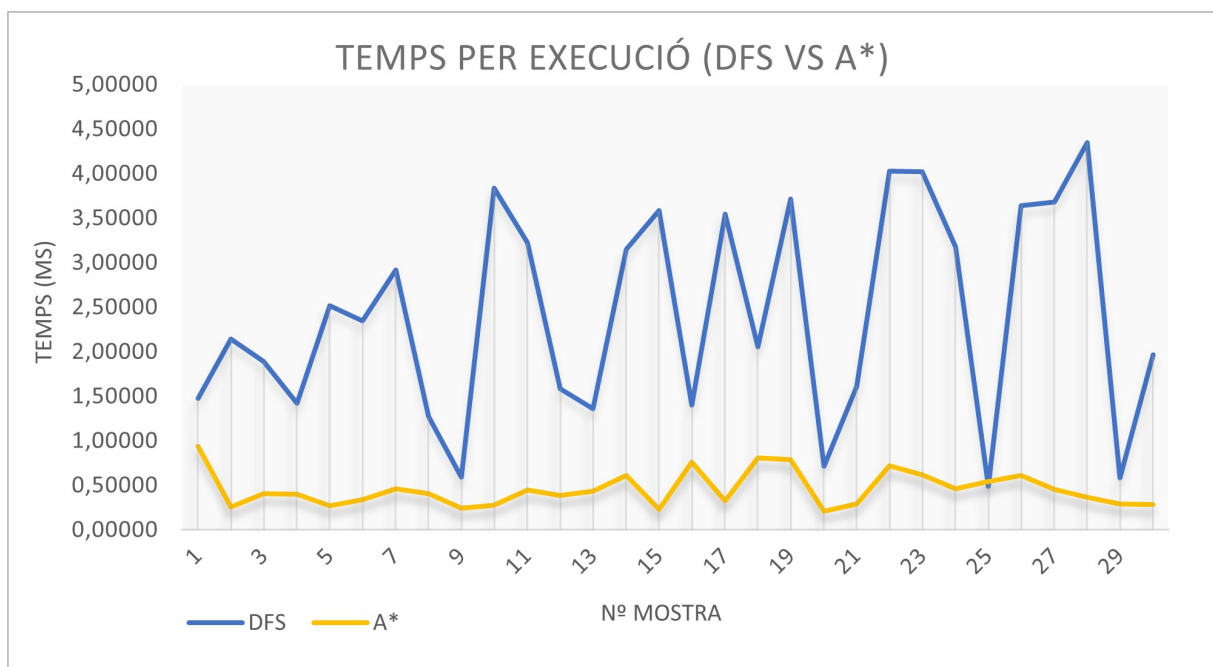


Figure 5.1: Gràfica comparativa: temps d'execució dels algorismes DFS i A*

En quant a l'algorisme minimax veim que, per qualche motiu que desconexim, hi ha execucions que tarden un temps desorbitat, això fa que tant la mitja de l'experiment com la desviació típica es vegin alterades.

A la següent gràfica es pot observar el succès

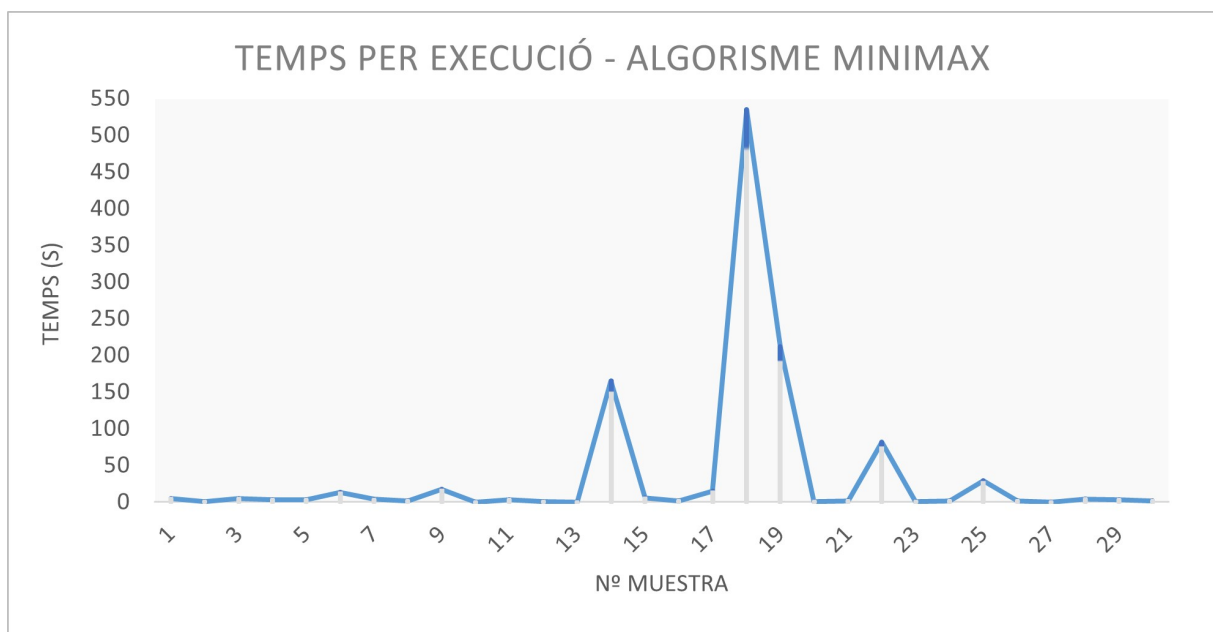


Figure 5.2: Gràfica comparativa: temps d'execució dels algorismes DFS i A*

Algorisme	Mitja	Desviació típica
DFS	0,002409387	0,001169715
A*	0,000454857	0,000192119
MINIMAX	37,13	104,1104874

Table 5.1: Taula comparativa: temps de execució

5.2 Mesura de rendiment: Nombre de passes

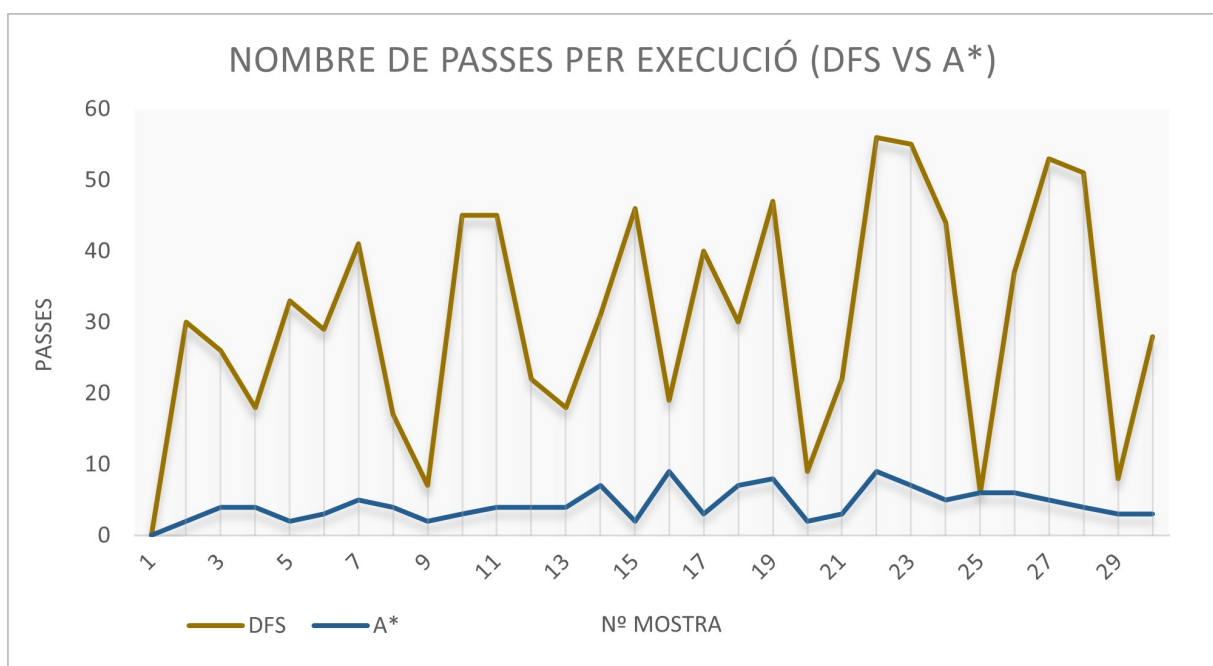


Figure 5.3: Gràfica comparativa: DFS i A*

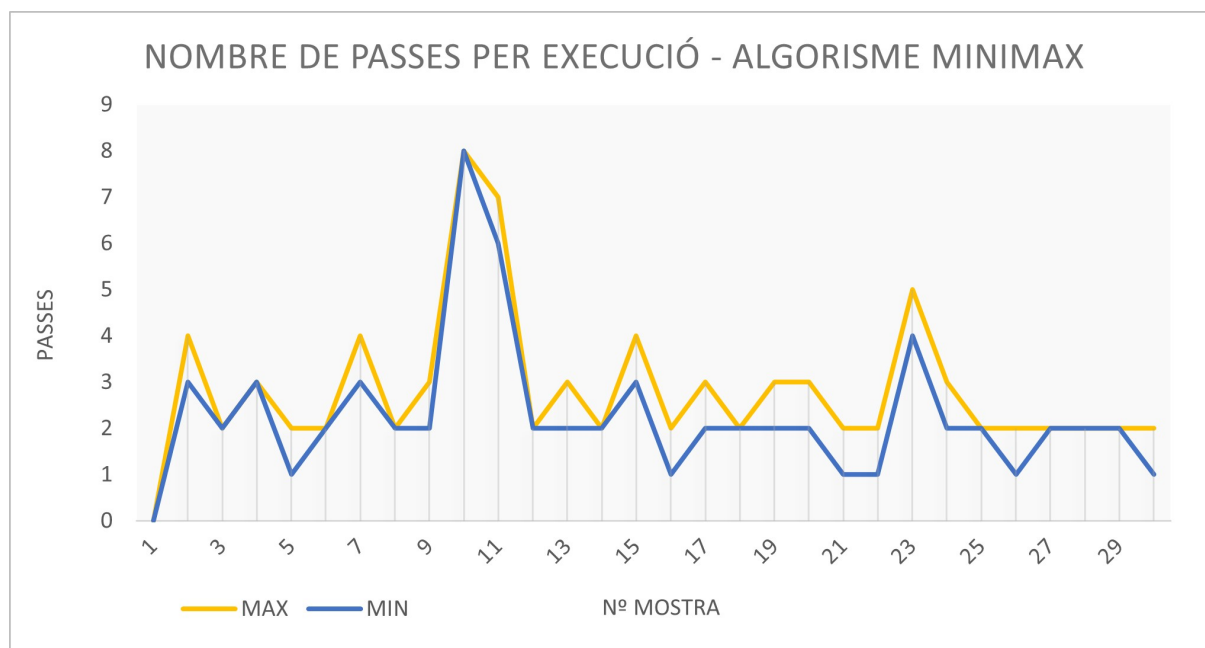


Figure 5.4: Gràfica de resultats: Algorisme Minimax

Algorisme	Mitja	Desviació típica
DFS	31,482758621	14,8964719
A*	4,482758621	2,061480714
MINIMAX (MAX)	2,93	1,48396102
MINIMAX (MIN)	2,34	1,468657892

Table 5.2: Taula comparativa: Nombre de passes per execució

Conclusió

La pràctica realitzada ha permès aplicar els conceptes i tècniques apresos al llarg del curs d'intel·ligència artificial per al desenvolupament d'agents basats en cerques i agents adversarials. Mitjançant la implementació dels algoritmes de cerca no informada, A* i Minimax amb poda α - β , hem pogut observar de primera mà com cada tècnica aborda el problema de trobar un camí cap a l'objectiu i com afecta la seva eficiència i rendiment segons les restriccions del taulell i les accions disponibles.

A més, la pràctica ha posat de manifest la importància de la representació adequada dels estats, la gestió de les accions i el càlcul de puntuacions en entorns multiagent. L'anàlisi comparatiu dels algorismes implementats ha evidenciat les diferències entre cerques informades i no informades, així com la rellevància de la poda α - β per optimitzar la presa de decisions en situacions adversarials.

En conclusió, aquesta pràctica no només ha reforçat els coneixements teòrics adquirits a classe, sinó que també ha desenvolupat habilitats de programació, anàlisi i documentació de solucions en un entorn controlat, proporcionant una base sòlida per a futurs treballs en intel·ligència artificial aplicada.

Pràctica Bloc I

Intel·ligència Artificial

Agents intel·ligents adversarials
i basats en cerca

Serafí Nebot Ginard
Jaume Galmés Ramis

Grau d'Enginyeria Informàtica
Curs 2025/2026